Lecture 24 → Cryptography and Security

3 actors

α Sender ("alpha")
β Receiver ("beta")
τ Adversary on the path ("tau")

What Can τ Do?

Attack	Threat	We need
Eavesdrop	read the message	**confidentiality**
Modify	change bits mid-flight	**integrity**
Impersonate	pretend to be the peer	**authentication**
Replay	resend old captured msg	**freshness**
Deny later	"I never sent that"	**non-repudiation**

Symmetric Cryptography

α and β share a secret key k:

$$c = E_k(m) \qquad m = D_k(c)$$

Workhorse: AES (NIST FIPS 197, 2001).

- 128-bit blocks; 128 / 192 / 256-bit keys.
- Hardware-accelerated on every modern CPU (AES-NI).
- ∼5 GB/s single-core — essentially free.

Block cipher modes:

- **ECB** — leaks patterns. *Never use.*
- **CBC** — confidentiality only, no integrity. Legacy.
- **GCM** — authenticated encryption (AEAD). Confidentiality **and** integrity in one primitive. **What TLS 1.3 uses.**

The Key Distribution Problem

Symmetric crypto needs a shared key. How do α and β agree on k when τ is watching the channel?

Three answers we will build to:

1. Pre-shared keys. Works. Doesn't scale
2. Asymmetric crypto to transport k. The classic TLS 1.2 pattern
3. Diffie–Hellman key agreement. The TLS 1.3 pattern — and the interesting one

Hash Functions

A hash function h : {0, 1} ∗ → {0, 1} n maps arbitrary-length input to a fixed-length digest.

SHA-256: 256-bit digest. No key. One-way compression

Three security properties:

1. Preimage resistance. Given y = h(m), hard to find any m′ with h(m′) = y.
2. Second-preimage resistance. Given m, hard to find m′ ≠ m with h(m′) = h(m)
3. Collision resistance. Hard to find any m ≠ m′ with h(m) = h(m′)

Uses: file integrity, password storage, commitment schemes, building blocks for MACs and signatures (coming up)

The Asymmetric Idea

Each party holds a key pair: (Kpub,Kpriv)

- Knowing the public key does not reveal the private key
- Encrypt with receiver's public key, decrypt with their private key → confidentiality
- Sign with sender's private key, verify with their public key → authentication, integrity, non-repudiation

Two operations that break everything else open. We spend the next 15 minutes on one specific construction: RSA.

Number Theory We Need

Euler's totient. $\varphi(n) =$ count of integers in $[1, n]$ coprime to n. For distinct primes p, q:

$$\varphi(pq) = (p-1)(q-1).$$

Euler's theorem. If $\gcd(a, n) = 1$:

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

RSA Key Generation

1. Pick two large primes p, q. (1024 bits each, in practice.)
2. Compute n = pq and φ(n) = (p − 1)(q − 1)
3. Pick public exponent e with gcd(e, φ(n)) = 1. In practice e = 65537
4. Compute d = e −1 (mod φ(n)) by the extended Euclidean algorithm

Public key	(n, e)
Private key	(n, d)

RSA Encryption and Decryption

Encode the message as an integer m with $0 \le m < n$.

$$\begin{aligned} \text{Encrypt:} \quad & c = m^e \bmod n \\ \text{Decrypt:} \quad & m = c^d \bmod n \end{aligned}$$

Why this works. We chose $ed \equiv 1 \pmod{\varphi(n)}$, so $ed = 1 + k\varphi(n)$ for some integer k. Then:

$$c^d = m^{ed} = m \cdot \left(m^{\varphi(n)}\right)^k \equiv m \cdot 1^k \equiv m \pmod{n}$$

by Euler's theorem.

Worked Example

Key generation.

- $p = 11$, $q = 13 \quad \Rightarrow \quad n = 143$, $\varphi(n) = 10 \cdot 12 = 120$.
- $e = 7$ (coprime to 120).
- $d = 103$ (check: $7 \cdot 103 = 721 = 6 \cdot 120 + 1$).

Encrypt $m = 9$**:**

$$c = 9^7 \bmod 143 = 4{,}782{,}969 \bmod 143 = 48.$$

Decrypt $c = 48$**:**

$$m = 48^{103} \bmod 143 = 9.$$

-

Why RSA Is Secure → An attacker with (n, e) who wants d needs φ(n) = (p − 1)(q − 1), which needs p and q — i.e., must factor n

Signing Is the Same Math, Flipped

For RSA:

$$\text{Sign:} \quad s = h(m)^d \bmod n \quad \text{(signer uses } \textit{private} \text{ exponent)}$$

$$\text{Verify:} \quad s^e \bmod n \stackrel{?}{=} h(m) \quad \text{(verifier uses } \textit{public} \text{ exponent)}$$

Why we hash first.

- Efficiency: one modular exponentiation signs arbitrary-length input.
- Security: textbook RSA on raw m has known forgery attacks (multiplicativity). Hash + padding (RSA-PSS) avoids them.

Diffie–Hellman Key Exchange

Public parameters: a large prime p and generator g. Anyone can know these.

Protocol:

1. α picks private a, sends A = g a mod p
2. β picks private b, sends B = g b mod p
3. α computes B a = g ab mod p
4. β computes A b = g ab mod p

Now α and β share g ab mod p without ever sending it. τ sees g a and g b on the wire. To recover g ab , τ must solve the discrete log problem — believed hard.

Why It Matters for TLS 1.3

Forward Secrecy

If β's long-term private key leaks tomorrow, recorded traffic from today is **still safe** — the session key was derived from ephemeral values that were discarded.
TLS 1.3 removed the old RSA key-transport mode entirely to guarantee this property.

Primitive to Requirement

Requirement	Primitive	Key kind
Confidentiality	AES-GCM	symmetric
Integrity	MAC or AEAD	symmetric
Authentication (peer)	signature or MAC	asym or sym
Authentication (message)	MAC	symmetric
Non-repudiation	**signature**	**asymmetric only**
Freshness	nonce / timestamp	protocol

MACs — Message Authentication Codes

Given a shared key k, prove:

- The message has not been modified (integrity).
- It came from someone who knows k (authentication).

HMAC (RFC 2104):

$$\mathrm{HMAC}_k(m) = h\big((k \oplus \mathrm{opad}) \,\|\, h((k \oplus \mathrm{ipad}) \,\|\, m)\big)$$

The two-level hash defends against attacks on naive $h(k\|m)$.

What HMAC does not give you: non-repudiation. Both α and β have k; either could have made the tag.

In modern TLS, MACs are subsumed by AEAD (AES-GCM), which rolls encryption and MAC into one primitive.

Digital Signatures

$$s = \mathrm{Sign}_{K_{\mathrm{priv}}}(h(m)) \qquad \mathrm{Verify}_{K_{\mathrm{pub}}}(s, h(m)) \in \{T, F\}$$

- Integrity: τ cannot change m without breaking verification.
- Authentication: only K_{priv} holder could have signed.
- **Non-repudiation:** K_{priv} holder can't deny it.

Sign the hash, not the message. Efficiency + security (padding schemes like RSA-PSS avoid textbook-RSA forgery attacks).

Modern alternatives. Ed25519 and ECDSA are much smaller than RSA for equivalent security:

- Ed25519 signature: $\sim$64 bytes
- RSA-2048 signature: 256 bytes

Most new systems default to Ed25519.

The Trust Problem

β's browser just received a public key claiming to be from `bank.com`. How does β know τ didn't generate that public key?

A public key, by itself, is just bits. It says nothing about *whose* key it is.

Answer: certificates. A trusted third party cryptographically binds the public key to an identity.

Certificates and CAs

A **certificate**:

$$\text{cert} = (\text{identity}, K_{\text{pub}}, \text{validity}, \ldots) + \text{signature by a CA}$$

A **Certificate Authority (CA)** is a trusted third party. Browsers and OSes ship a **root store** — a preinstalled list of root CA public keys.

Chain of trust:

$$\text{leaf cert} \xrightarrow{\text{signed by}} \text{intermediate CA} \xrightarrow{\text{signed by}} \text{root CA}$$

β's browser walks the chain from leaf to any root in its store.

- **Roots:** hundreds globally.
- **Intermediates:** many.
- **Leaves:** one per domain.

Real-World PKI

Let's Encrypt changed everything.

- Free, automated (ACME protocol), launched 2016.
- HTTPS share of web traffic: $\sim$30% in 2015 $\rightarrow$ $\sim$95% today.
- Made "HTTPS by default" economically obvious.

When CAs go bad:

- **DigiNotar (2011)** — compromised, issued fraudulent `*.google.com` certs, removed from every root store. Company bankrupt within weeks.
- **Symantec (2017–18)** — Google progressively distrusted Symantec's CA hierarchy after mis-issuance.

Certificate pinning — apps "pin" to a specific expected cert or public key. Defense in depth at the cost of operational fragility.

Live: Inspect a Real Cert Chain

```
openssl s_client -connect bu.edu:443 -servername bu.edu < /dev/null
openssl s_client -connect expired.badssl.com:443 \
    -servername expired.badssl.com
```

TLS 1.3 Handshake

1. ClientHello. α sends supported ciphers and an ephemeral DH public share g a
2. ServerHello. β picks a cipher and sends: its own ephemeral DH share g b , its certificate chain, a signature over the handshake transcript
3. α verifies: cert chain against root store, then β's signature
4. Both derive session keys. Compute g ab, run through HKDF, get AES-GCM keys
5. Application data. Encrypted with AES-GCM

Every Primitive, One Handshake

Step	Primitive	From
Key share	Diffie–Hellman (ephemeral)	§DH
Cert chain verify	Digital signatures + PKI	§Signatures, PKI
Handshake auth	Signature on transcript	§Signatures
Bulk encryption	AES-GCM (AEAD)	§Symmetric
Bulk integrity	AES-GCM (AEAD)	§Symmetric
Key derivation	HKDF (hash-based)	§Hash

Forward Secrecy, Revisited

Session keys come from *ephemeral* DH values that are discarded at the end of the session.

Consequence: even if β's long-term signing key is compromised tomorrow, traffic recorded today cannot be decrypted.

The long-term key only signs handshakes — it never encrypts anything.

Store-now-decrypt-later

Adversaries are recording encrypted traffic today, betting that tomorrow's cryptography (or tomorrow's key compromise) will let them decrypt it. Forward secrecy is the defense.

Loop Back to L23: QUIC

What "built in" means: QUIC integrates the TLS 1.3 handshake into the transport-layer handshake itself.

- One round-trip establishes connection **and** crypto.
- 0-RTT resumption sends application data in the first packet on return visits.
- Streams are encrypted independently — even packet headers are partially encrypted.

The promise from L23, delivered: QUIC closes the full loop. Every layer, encrypted by default, in one round-trip.